

QA Evidence Bundle — nezha overnight session (2026-06-16)

§11.4.83 consolidated end-to-end QA evidence bundle. This is a *pointer-based index* into already-committed captured evidence from the 2026-06-16 overnight session run on the dedicated heavy-testing node `nezha.local`. Every claim below cites a real, verified file path or commit SHA. No evidence is re-derived here; this file aggregates what was already proven and recorded.

Run-id: 2026-06-16-nezha-overnight **Host:** `nezha.local` (i7/8-core, 64 GB, NVMe, `/dev/kvm` live, rootless Podman 5.7.1 + `podman-compose` 1.5.0) **Verified-against HEAD at bundle authoring:** `d256ed1a` **Anti-bluff posture:** every shipped item carries a falsifiability rehearsal (Bluff-Audit stamp) captured in the commit body. Deferred items are marked NOT-YET-PROVEN — never as done.

Cross-reference: closure discipline (§6.O / §6.T.4)

- **§6.T.4 (Bugfix Documentation):** the provider fixes are recorded in `docs/issues/fixed/BUGFIXES.md` (verified present; entries dated 2026-06-16 for `tokyotosho` + the 6 sibling providers).
 - **§6.O (Crashlytics-Resolved Coverage):** N/A this session — the fixes are server-side Go provider/test-infra fixes surfaced by `nezha` real-tracker E2E and by `ENOSPC` during a hermetic-test run, not Crashlytics-recorded client crashes. No `.lava-ci-evidence/crashlytics-resolved/` entry is owed for these.
-

1. nezha whole-System boot + real-HTTP verification — PASS

What was tested: distributed boot of the whole Lava System (10 containers: prod + dev lava-api-go, 2 Postgres, Prometheus/Loki/Tempo/Grafana, Jaxx, FlareSolverr) on nezha.local, built natively (x86_64) from the synced repo, with real HTTP verification at the same surfaces a user/orchestrator touches (\$6.B — not podman ps).

Captured-evidence artifact: .lava-ci-evidence/nezha/2026-06-16-system-boot.md (verified present).

- Real podman ps inventory (2026-06-16T16:08:17Z) — all 10 Up.
- Real HTTP: prod /health {"status":"alive"}, /ready {"status":"ready"}, /providers = 13 providers, version lava-api-go 2.3.30 (build 2330); dev /health alive; Prometheus + Grafana healthy. golang-migrate v9 head on both DBs.
- 4 boot issues root-caused (evidence, not guess) and fixed in the same doc: incomplete submodule init; api-go crash-loop (dynamic-link / missing CGO_ENABLED=0); TLS permission denied (cert perms under rootless users); co-tenant port collisions.

Commit: d6f8fdad — infra(nezha): boot whole Lava System on nezha.local ... (verified present). **Status:** PASS (real-HTTP verified at the live surfaces).

2. Heavy Go suite — 49 ok / 0 fail — PASS

What was tested: the full lava-api-go heavy test suite (e2e / contract / parity / integration) against real podman Postgres on nezha, plus realtrackers across 20 providers.

Captured-evidence pointer: docs/CONTINUATION.md:15 (verified present) records "heavy Go suite **49 ok / 0 fail** (real podman Postgres e2e/contract/parity/ integration); realtrackers 20 providers green" as the Phase 1-4 DONE state.

Status: PASS per the recorded CONTINUATION state. **Honesty note:** the 49-ok count is recorded in docs/CONTINUATION.md; a standalone per-suite captured log file under .lava-ci-evidence/nezha/ was not located for this bundle. The count is cited from CONTINUATION, not re-derived.

3. 10-provider retry resilience (bounded-retry on transient upstream failure) — PASS

What was tested: transient-failure handling (network/timeout or 5xx) on the HTTP-backed providers. nezha real-tracker E2E surfaced the gap on Tokyo Toshokan (live test PASSED 16:14 / FAILED 16:15 on a slow upstream → single unretried attempt produced a user-facing "unknown error"). The fix was then propagated to every sibling sharing the single-attempt class, completing all 10 HTTP providers.

Captured-evidence artifacts:

- docs/issues/fixed/BUGFIXES.md (verified present) — 2026-06-16 entries: "tokyotosho curated provider: user-facing 'unknown error' on slow upstream" (root cause + affected files + verification test) and "6 sibling providers shared the tokyotosho single-attempt-no-retry gap".

Commits + falsifiability rehearsals (Bluff-Audit stamps, all verified in commit bodies):

- a88467df — tokyotosho bounded retry. Mutation: bypass the retry loop in fetchFeed. Observed-Failure: TestSearch_RetriesOnTransient5xx FAIL ("...recover via retry: tokyotosho: HTTP 503: provider: unknown error"). Reverted: yes.
- cd54341c — propagate to 6 siblings (knaben, nyaa, bitsearch, torrentdownloads, gutenber, flaresolverr). Mutation: bypass the retry loop. Observed-Failure: retry test FAIL, all 6 ("...recover via retry: knaben: HTTP 503: ...unknown error"). Reverted: yes (all 6).
- 307b4a5d — retry-inside-breaker for kinozal/nnmclub/rutracker (a transient timeout/5xx had been counting as a breaker failure). Mutation: bypass the in-closure retry. Observed-Failure: TestRetriesOnTransient5xx FAIL, all 3; load-bearing checks GetFailures()==0 + StateClosed. Reverted: yes (all 3).

Status: PASS — "ALL 10 HTTP providers" fixed (per 307b4a5d body) with a falsifiability-proven regression test per provider.

4. Repo-pollution cleanup + hermetic test-infra fix (§6.J) — PASS

What was tested: the check-constitution hermetic tests used unguarded `f=$(mktemp -d)`; under ENOSPC `mktemp` returns empty → `git -C ""` operated on the REAL repo, planting 19 stray fixture commits into local master (cleaned; never pushed — origin stayed clean). The fix introduces a guarded `_safe_tmpdir()` (FATAL abort if `mktemp` fails or returns a non-absolute path) replacing every `$(mktemp -d)`.

Captured-evidence artifact: commit d256ed1a body (verified present) — full root-cause + fix + captured falsifiability.

Falsifiability rehearsal (captured in d256ed1a body): re-ran all 8 tests; ENOSPC recurred *during* the run, and HEAD stayed 307b4a5d with ZERO new commits

- ZERO planted files — the guard fired (FATAL refusal instead of pollution), while the 6 tests that ran before disk-fill passed. Pre-fix this exact ENOSPC produced the 19-commit pollution; post-fix it produces a clean refusal. Classification: project-specific.

Commit: d256ed1a —

`fix(tests):` guard hermetic-test `mktemp` against ENOSPC repo-pollution (§6.J test-infra bug). **Status:** PASS. **Honesty note:** this test-infra fix is documented in the d256ed1a commit body (the authoritative record), not in `BUGFIXES.md`; `BUGFIXES.md` covers the §3 provider fixes.

DEFERRED / NOT-YET-PROVEN (no bluff)

These items are NOT claimed as passing in this session. They remain blocked.

- **Phase 5 — emulator/Challenge matrix on nezha:** NOT-YET-PROVEN, **nezha-load blocked**. The dedicated node was under co-tenant load (shared host; see `.lava-ci-evidence/nezha/2026-06-16-system-boot.md` §4 port-collisions) and the authoring host hit ENOSPC mid-session (see `2026-06-16-releasable-state-sweep.md` — multiple Bash calls failed at the harness layer with ENOSPC). No per-AVD attestation rows were produced. Per §6.AE/§6.I a green matrix is a SET of per-AVD rows; absent those, this is NOT proven.

- **Phase 8 — Firebase redistribute:** NOT-YET-PROVEN, §6.Z-gated.
No artifact is distributed in this session. §6.Z forbids distributing without the Compose UI Challenge Tests EXECUTED against the exact artifact; that execution did not occur this session (Phase 5 blocked, above). Redistribute remains held.
- **Gate-health sweep (releasable-state):** `.lava-ci-evidence/nezha/2026-06-16-releasable-state-sweep.md` (verified present) is an HONEST read-only audit that recorded gate FAILs (e.g. non-fatal-coverage §6.AC, markdown-export §11.4.65, coverage-ledger §11.4.25, several hermetic suites) with advisory-vs-strict status partly UNKNOWN because ENOSPC prevented the sweep from printing its overall verdict. These are recorded as findings, NOT as passing. This bundle does not claim the gate sweep is green.

Evidence inventory (all paths/SHAs verified present at authoring time)

Ref	Path / SHA	Verified
nezha boot	<code>.lava-ci-evidence/nezha/2026-06-16-system-boot.md</code>	yes (ls)
gate sweep	<code>.lava-ci-evidence/nezha/2026-06-16-releasable-state-sweep.md</code>	yes (ls)
bugfixes (§6.T.4)	<code>docs/issues/fixed/BUGFIXES.md</code>	yes (ls + grep entries)
heavy Go 49 ok	<code>docs/CONTINUATION.md:15</code>	yes (sed)
boot commit	<code>d6f8fdad</code>	yes (git log -1)
tokyotosho retry	<code>a88467df</code>	yes (git log -1 + Bluff-Audit)
6-sibling retry	<code>cd54341c</code>	yes (git log -1 + Bluff-Audit)
breaker retry (3)	<code>307b4a5d</code>	yes (git log -1 + Bluff-Audit)
ENOSPC test- infra	<code>d256ed1a</code>	yes (git log -1 + full body)